

```

//
=====
=====
// [시각/치수 100% 완전 일치본] physics.js - 전체 소스 코드
//
=====
=====

const canvas = document.getElementById('simCanvas');
const ctx = canvas.getContext('2d');

let dprWidth = 0;
let dprHeight = 0;

// [통일된 월드 공간 스케일 세팅]
const MAX_WORLD_X = 180; // 최대 전진 거리 180m
const MAX_WORLD_Y = 30; // 최대 높이 30m
const MAX_WORLD_Z = 15; // 좌우 최대 관측 편차 범위 (±15m)
const TARGET_SLANT_R = 145; // 사대 0점부터 과녁 바닥 전면까지의 고정 경사 거리
(145m 부동)

// 국궁 표준 과녁 물리 제원 및 수직 투영 보정
const TGT_W = 2.0; // 가로 2m
const TGT_H = 2.667; // 실제 사선 세로 길이 2.667m
const TGT_D = 0.5; // 두께 0.5m
const TGT_TILT = 15 * Math.PI / 180; // 뒤로 15도 기울어짐 (라디안)
const TGT_PROJ_H = TGT_H * Math.cos(TGT_TILT); // 정면/측면에서 보이는 수직 투영 높
이 = 정확히 2.58m

function resizeCanvas() {
  const rect = canvas.getBoundingClientRect();
  canvas.width = rect.width * window.devicePixelRatio;
  canvas.height = rect.height * window.devicePixelRatio;
  ctx.resetTransform();
  ctx.scale(window.devicePixelRatio, window.devicePixelRatio);
  dprWidth = rect.width;
  dprHeight = rect.height;
  drawScene();
}
window.addEventListener('resize', resizeCanvas);
window.addEventListener('load', resizeCanvas);

let isFlying = false;

```

```

let animationFrameId = null;
let trajectory = [];

let arrowState = { x: 0, y: 0, z: 0, vx: 0, vy: 0, vz: 0, pitch: 0, yaw: 0 };

let flightMetrics = {
  maxDistance: 0,
  maxHeight: 0,
  sideDeviation: 0,
  flightTime: 0,
  impactVelocity: 0,
  impactEnergy: 0
};

let targetHitMetrics = {
  isHit: false,
  localZ: 0,
  localY: 0
};

let hasReachedTargetX = false;
let hasReachedTargetY = false;
let hasIntersectedTargetPlane = false;

const ORIGIN_X_OFFSET = 35;
const GROUND_Y_OFFSET = 30;

// 과녁 고도차를 '과녁 바닥'의 높이로 취급
function getDynamicTargetGeometry() {
  const targetHInput = parseFloat(document.getElementById('targetHeight').value);
  const targetH = isNaN(targetHInput) ? 0 : targetHInput;
  const safeTargetH = Math.min(targetH, TARGET_SLANT_R - 0.1);
  const targetBaseX = Math.sqrt(Math.pow(TARGET_SLANT_R, 2) -
Math.pow(safeTargetH, 2));
  return { baseX: targetBaseX, height: safeTargetH };
}

function fireArrow() {
  if (isFlying) cancelAnimationFrame(animationFrameId);
  if (typeof saveSettings === 'function') saveSettings();

  const v0 = parseFloat(document.getElementById('velocity').value) || 50;
  const angleDeg = parseFloat(document.getElementById('angle').value) || 0;

```

```

const yawDeg = parseFloat(document.getElementById('yawAngle').value) || 0;
const launchH = parseFloat(document.getElementById('launchHeight').value) || 1.5;
const launchZ = parseFloat(document.getElementById('launchZ').value) || 0;

const pitchRad = (angleDeg * Math.PI) / 180;
const yawRad = (yawDeg * Math.PI) / 180;

arrowState.x = 0;
arrowState.y = launchH;
arrowState.z = launchZ;

arrowState.vx = v0 * Math.cos(pitchRad) * Math.cos(yawRad);
arrowState.vy = v0 * Math.sin(pitchRad);
arrowState.vz = v0 * Math.cos(pitchRad) * Math.sin(yawRad);

arrowState.pitch = pitchRad;
arrowState.yaw = yawRad;

flightMetrics = { maxDistance: 0, maxHeight: launchH, sideDeviation: 0, flightTime: 0,
impactVelocity: v0, impactEnergy: 0 };
targetHitMetrics = { isHit: false, localZ: 0, localY: 0 };
hasReachedTargetX = false;
hasReachedTargetY = false;
hasIntersectedTargetPlane = false;

updateResultUI();
trajectory = [{ x: arrowState.x, y: arrowState.y, z: arrowState.z }];
isFlying = true;
animate();
}

function animate() {
  if (!isFlying) return;

  const cd = parseFloat(document.getElementById('dragCoeff').value) || 0;
  const cl = parseFloat(document.getElementById('liftCoeff').value) || 0;
  const d = (parseFloat(document.getElementById('diameter').value) || 5.5) / 1000;
  const m = (parseFloat(document.getElementById('weight').value) || 25) / 1000;
  const rho = parseFloat(document.getElementById('airDensity').value) || 1.225;
  const windX = parseFloat(document.getElementById('windX').value) || 0;
  const windZ = parseFloat(document.getElementById('windY').value) || 0;

  const g = 9.81; const dt = 0.016; const area = Math.PI * Math.pow(d / 2, 2);

```

```

const tgtGeo = getDynamicTargetGeometry();
const targetBaseX = tgtGeo.baseX; const targetH = tgtGeo.height;

const relVx = arrowState.vx - windX; const relVy = arrowState.vy; const relVz =
arrowState.vz - windZ;
const vRel = Math.sqrt(relVx * relVx + relVy * relVy + relVz * relVz) || 0.001;

const flowPitch = Math.atan2(relVy, Math.sqrt(relVx * relVx + relVz * relVz));
const flowYaw = Math.atan2(relVz, relVx);
const attackAngle = arrowState.pitch - flowPitch;

const effectiveArea = area * 2.5;
const dynamicLiftCoeff = 2.0 * Math.sin(attackAngle) * Math.cos(attackAngle);
const dragF = 0.5 * rho * vRel * vRel * cd * effectiveArea;
const liftF = 0.5 * rho * vRel * vRel * (cl + dynamicLiftCoeff) * effectiveArea;

const dragAx = (-dragF * Math.cos(flowPitch) * Math.cos(flowYaw)) / m;
const dragAy = (-dragF * Math.sin(flowPitch)) / m;
const dragAz = (-dragF * Math.cos(flowPitch) * Math.sin(flowYaw)) / m;
const liftAx = (-liftF * Math.sin(flowPitch) * Math.cos(flowYaw)) / m;
const liftAy = (liftF * Math.cos(flowPitch)) / m;
const liftAz = (-liftF * Math.sin(flowPitch) * Math.sin(flowYaw)) / m;

const ax = dragAx + liftAx; const ay = -g + dragAy + liftAy; const az = dragAz +
liftAz;
const prevX = arrowState.x; const prevY = arrowState.y; const prevZ = arrowState.z;

arrowState.vx += ax * dt; arrowState.vy += ay * dt; arrowState.vz += az * dt;
arrowState.x += arrowState.vx * dt; arrowState.y += arrowState.vy * dt; arrowState.z
+= arrowState.vz * dt;

arrowState.pitch = Math.atan2(arrowState.vy, Math.sqrt(arrowState.vx * arrowState.vx
+ arrowState.vz * arrowState.vz));
arrowState.yaw = Math.atan2(arrowState.vz, arrowState.vx);
trajectory.push({ x: arrowState.x, y: arrowState.y, z: arrowState.z });

if (!hasReachedTargetX) { flightMetrics.flightTime += dt; }
if (arrowState.y > flightMetrics.maxHeight) { flightMetrics.maxHeight = arrowState.y;
}

const nx = Math.cos(TGT_TILT); const ny = -Math.sin(TGT_TILT);
const distPrev = nx * (prevX - targetBaseX) + ny * (prevY - targetH);
const distCurr = nx * (arrowState.x - targetBaseX) + ny * (arrowState.y - targetH);

```

```

if (!hasIntersectedTargetPlane && distPrev * distCurr <= 0 && prevX < arrowState.x)
{
    hasIntersectedTargetPlane = true;
    const s = Math.abs(distPrev) / (Math.abs(distPrev) + Math.abs(distCurr));
    const interY = prevY + (arrowState.y - prevY) * s;
    const interZ = prevZ + (arrowState.z - prevZ) * s;
    const centerWorldY = targetH + (TGT_H / 2) * Math.cos(TGT_TILT);

    targetHitMetrics.localZ = interZ;
    targetHitMetrics.localY = (interY - centerWorldY) / Math.cos(TGT_TILT);

    if (Math.abs(targetHitMetrics.localZ) <= TGT_W / 2 &&
Math.abs(targetHitMetrics.localY) <= TGT_H / 2) {
        targetHitMetrics.isHit = true;
    } else {
        targetHitMetrics.isHit = false;
    }
}

if (!hasReachedTargetX && arrowState.x >= targetBaseX) { hasReachedTargetX =
true; }
if (!hasReachedTargetY && arrowState.vy <= 0 && prevY >= targetH &&
arrowState.y <= targetH) {
    hasReachedTargetY = true;
    const t = (prevY - targetH) / (prevY - arrowState.y);
    flightMetrics.maxDistance = prevX + (arrowState.x - prevX) * t;
    flightMetrics.sideDeviation = prevZ + (arrowState.z - prevZ) * t;
    const vFinal = Math.sqrt(arrowState.vx * arrowState.vx + arrowState.vy *
arrowState.vy + arrowState.vz * arrowState.vz);
    flightMetrics.impactVelocity = vFinal; flightMetrics.impactEnergy = 0.5 * m *
vFinal * vFinal;
}

if (!hasReachedTargetY) {
    flightMetrics.maxDistance = arrowState.x; flightMetrics.sideDeviation =
arrowState.z;
    const vCurrent = Math.sqrt(arrowState.vx * arrowState.vx + arrowState.vy *
arrowState.vy + arrowState.vz * arrowState.vz);
    flightMetrics.impactVelocity = vCurrent; flightMetrics.impactEnergy = 0.5 * m *
vCurrent * vCurrent;
}

```

```

    updateResultUI();
    if (arrowState.y <= 0) { arrowState.y = 0; isFlying = false; updateResultUI(); }
    if (arrowState.x > MAX_WORLD_X || arrowState.x < -10) { isFlying = false; }

    drawScene();
    if (isFlying) { animationFrameId = requestAnimationFrame(animate); }
}

function updateResultUI() {
    const resDist = document.getElementById('resMaxDist'); const resHeight =
document.getElementById('resMaxHeight');
    const resSide = document.getElementById('resSideDev'); const resTime =
document.getElementById('resFlightTime');
    const resVel = document.getElementById('resImpactVel'); const resEnergy =
document.getElementById('resImpactEnergy');
    if (resDist) resDist.innerText = flightMetrics.maxDistance.toFixed(2) + " m";
    if (resHeight) resHeight.innerText = flightMetrics.maxHeight.toFixed(2) + " m";
    if (resSide) resSide.innerText = flightMetrics.sideDeviation.toFixed(2) + " m";
    if (resTime) resTime.innerText = flightMetrics.flightTime.toFixed(2) + " s";
    if (resVel) resVel.innerText = flightMetrics.impactVelocity.toFixed(2) + " m/s";
    if (resEnergy) resEnergy.innerText = flightMetrics.impactEnergy.toFixed(2) + " J";
}

function drawScene() {
    if (dprWidth === 0 || dprHeight === 0) return;
    ctx.clearRect(0, 0, dprWidth, dprHeight);

    const tgtGeo = getDynamicTargetGeometry();
    const targetBaseX = tgtGeo.baseX; const safeTargetH = tgtGeo.height;
    const availW = dprWidth - ORIGIN_X_OFFSET - 10; const availH = dprHeight -
GROUND_Y_OFFSET - 10;

    const scaleX = availW / MAX_WORLD_X; const scaleY = availH / MAX_WORLD_Y;
    const topScaleForward = (dprHeight - 40) / MAX_WORLD_X;
    const topScaleSide = (dprWidth - 20) / (MAX_WORLD_Z * 2);
    const frontScaleZ = (dprWidth - ORIGIN_X_OFFSET - 20) / (MAX_WORLD_Z * 2);
    const frontScaleY = availH / MAX_WORLD_Y;
    const targetViewScale = Math.min(dprWidth, dprHeight) / 5.5;

    function toScreen(pX, pY, pZ) {
        if (currentView === 'side') {
            return { x: ORIGIN_X_OFFSET + (pX * scaleX), y: dprHeight - GROUND_Y_OFFSET -
(pY * scaleY) };

```

```

    }
    if (currentView === 'top') {
        return { x: (dprWidth / 2) + (pZ * topScaleSide), y: dprHeight - 25 - (pX *
topScaleForward) };
    }
    if (currentView === 'front') {
        return { x: ORIGIN_X_OFFSET + (dprWidth - ORIGIN_X_OFFSET - 20) / 2 + (pZ *
frontScaleZ), y: dprHeight - GROUND_Y_OFFSET - (pY * frontScaleY) };
    }
    if (currentView === 'target') {
        return { x: (dprWidth / 2) + (pZ * targetViewScale), y: (dprHeight * 0.65) - (pY *
targetViewScale) };
    }
    return { x: 0, y: 0 };
}

// 눈금선 기본 스타일 레이아웃 설정
ctx.strokeStyle = '#e5e5ea';
ctx.lineWidth = 1;
ctx.font = '10px -apple-system';
ctx.fillStyle = '#8e8e93';

// 1. 측면도 (Side View) 격자 렌더링
if (currentView === 'side') {
    for (let xMeters = 0; xMeters <= MAX_WORLD_X; xMeters += 20) {
        let scrX = ORIGIN_X_OFFSET + (xMeters * scaleX);
        ctx.beginPath(); ctx.moveTo(scrX, 0); ctx.lineTo(scrX, dprHeight - GROUND_Y_OFFSET);
ctx.stroke();
        ctx.textAlign = 'center'; ctx.fillText(xMeters + 'm', scrX, dprHeight -
GROUND_Y_OFFSET + 14);
    }
    for (let yMeters = 0; yMeters <= MAX_WORLD_Y; yMeters += 5) {
        let scrY = dprHeight - GROUND_Y_OFFSET - (yMeters * scaleY);
        ctx.beginPath(); ctx.moveTo(ORIGIN_X_OFFSET, scrY); ctx.lineTo(dprWidth, scrY);
ctx.stroke();
        ctx.textAlign = 'right'; ctx.fillText(yMeters + 'm', ORIGIN_X_OFFSET - 5, scrY + 3);
    }
}
// 메인 기선축
ctx.strokeStyle = '#1d1d1f'; ctx.lineWidth = 2;
ctx.beginPath(); ctx.moveTo(ORIGIN_X_OFFSET, dprHeight - GROUND_Y_OFFSET);
ctx.lineTo(ORIGIN_X_OFFSET + (MAX_WORLD_X * scaleX), dprHeight -
GROUND_Y_OFFSET); ctx.stroke();
ctx.beginPath(); ctx.moveTo(ORIGIN_X_OFFSET, 0); ctx.lineTo(ORIGIN_X_OFFSET,

```

```
dprHeight - GROUND_Y_OFFSET); ctx.stroke();
```

```
// 2. 조감도 (Top View) 격자 렌더링
```

```
} else if (currentView === 'top') {  
  for (let xMeters = 0; xMeters <= MAX_WORLD_X; xMeters += 20) {  
    let scrY = dprHeight - 25 - (xMeters * topScaleForward);  
    ctx.beginPath(); ctx.moveTo(0, scrY); ctx.lineTo(dprWidth, scrY); ctx.stroke();  
    ctx.textAlign = 'left'; ctx.fillText(xMeters + 'm', 8, scrY - 4);  
  }  
  for (let zMeters = -MAX_WORLD_Z; zMeters <= MAX_WORLD_Z; zMeters += 5) {  
    let scrX = (dprWidth / 2) + (zMeters * topScaleSide);  
    ctx.beginPath(); ctx.moveTo(scrX, 0); ctx.lineTo(scrX, dprHeight); ctx.stroke();  
    ctx.textAlign = 'center'; ctx.fillText(zMeters === 0 ? '중앙(0m)' : zMeters + 'm', scrX,  
dprHeight - 8);  
  }  
  // 사대 중앙 분리 기준축선  
  ctx.strokeStyle = '#86868b'; ctx.lineWidth = 1.5;  
  ctx.beginPath(); ctx.moveTo(dprWidth / 2, 0); ctx.lineTo(dprWidth / 2, dprHeight - 25);  
  ctx.stroke();  
}
```

```
// 3. 정면도 (Front View) 격자 렌더링
```

```
} else if (currentView === 'front') {  
  const centerX = ORIGIN_X_OFFSET + (dprWidth - ORIGIN_X_OFFSET - 20) / 2;  
  for (let zMeters = -MAX_WORLD_Z; zMeters <= MAX_WORLD_Z; zMeters += 5) {  
    let scrX = centerX + (zMeters * frontScaleZ);  
    ctx.beginPath(); ctx.moveTo(scrX, 0); ctx.lineTo(scrX, dprHeight - GROUND_Y_OFFSET);  
    ctx.stroke();  
    ctx.textAlign = 'center'; ctx.fillText(zMeters === 0 ? '0m' : zMeters + 'm', scrX,  
dprHeight - GROUND_Y_OFFSET + 14);  
  }  
  for (let yMeters = 0; yMeters <= MAX_WORLD_Y; yMeters += 5) {  
    let scrY = dprHeight - GROUND_Y_OFFSET - (yMeters * frontScaleY);  
    ctx.beginPath(); ctx.moveTo(ORIGIN_X_OFFSET, scrY); ctx.lineTo(dprWidth, scrY);  
    ctx.stroke();  
    ctx.textAlign = 'right'; ctx.fillText(yMeters + 'm', ORIGIN_X_OFFSET - 5, scrY + 3);  
  }  
  // 지면 및 좌측 기준 기선  
  ctx.strokeStyle = '#1d1d1f'; ctx.lineWidth = 2;  
  ctx.beginPath(); ctx.moveTo(ORIGIN_X_OFFSET, dprHeight - GROUND_Y_OFFSET);  
  ctx.lineTo(dprWidth, dprHeight - GROUND_Y_OFFSET); ctx.stroke();  
  ctx.beginPath(); ctx.moveTo(ORIGIN_X_OFFSET, 0); ctx.lineTo(ORIGIN_X_OFFSET,  
dprHeight - GROUND_Y_OFFSET); ctx.stroke();  
  // 정가운데 수직 기준선
```



```

    ctx.strokeStyle = '#86868b'; ctx.lineWidth = 1.2;
    ctx.beginPath(); ctx.moveTo(centerX, 0); ctx.lineTo(centerX, dprHeight -
GROUND_Y_OFFSET); ctx.stroke();

// 4. 과녁 확대도 (Target View) 정밀 격자 렌더링
} else if (currentView === 'target') {
    const tBottomY = dprHeight * 0.65;
    // 색상 전이 현상 방지를 위해 격자선 컬러 명시적 재할당 및 두께 미세화
    ctx.strokeStyle = '#e5e5ea';
    ctx.lineWidth = 0.8;

    for (let lz = -2.0; lz <= 2.0; lz += 0.5) {
        let scrX = (dprWidth / 2) + (lz * targetViewScale);
        ctx.beginPath(); ctx.moveTo(scrX, 0); ctx.lineTo(scrX, dprHeight); ctx.stroke();
        ctx.fillStyle = '#8e8e93'; ctx.font = '9px -apple-system'; ctx.textAlign = 'center';
        ctx.fillText(lz === 0 ? '중앙' : lz.toFixed(1) + 'm', scrX, dprHeight - 8);
    }
    for (let ly = -0.5; ly <= 3.0; ly += 0.5) {
        let scrY = tBottomY - (ly * targetViewScale);
        ctx.beginPath(); ctx.moveTo(0, scrY); ctx.lineTo(dprWidth, scrY); ctx.stroke();
        ctx.fillStyle = '#8e8e93'; ctx.font = '9px -apple-system'; ctx.textAlign = 'right';
        ctx.fillText(ly === 0 ? '바닥(0m)' : (ly > 0 ? '+' : '-') + ly.toFixed(1) + 'm', dprWidth - 8,
scrY + 3);
    }
    // 과녁 조준 십자 기선축
    ctx.strokeStyle = '#86868b'; ctx.lineWidth = 1.2;
    ctx.beginPath(); ctx.moveTo(dprWidth / 2, 0); ctx.lineTo(dprWidth / 2, dprHeight);
ctx.stroke();
    ctx.beginPath(); ctx.moveTo(0, tBottomY); ctx.lineTo(dprWidth, tBottomY); ctx.stroke();
}

ctx.lineWidth = 1.5;
// 과녁 객체 드로잉
if (currentView === 'side') {
    const fBottom = toScreen(targetBaseX, safeTargetH, 0);
    const frontTopX = targetBaseX + TGT_H * Math.sin(TGT_TILT);
    const frontTopY = safeTargetH + TGT_PROJ_H;
    const fTop = toScreen(frontTopX, frontTopY, 0);
    const thickX = TGT_D * Math.cos(TGT_TILT);
    const thickY = -TGT_D * Math.sin(TGT_TILT);
    const bBottom = toScreen(targetBaseX + thickX, safeTargetH + thickY, 0);
    const bTop = toScreen(frontTopX + thickX, frontTopY + thickY, 0);

```

```

    ctx.fillStyle = '#e5e5ea';
    ctx.beginPath(); ctx.moveTo(fBottom.x, fBottom.y); ctx.lineTo(fTop.x, fTop.y);
    ctx.lineTo(bTop.x, bTop.y); ctx.lineTo(bBottom.x, bBottom.y); ctx.closePath(); ctx.fill();
    ctx.strokeStyle = '#1d1d1f'; ctx.lineWidth = 1; ctx.stroke();
    ctx.strokeStyle = '#1d1d1f'; ctx.lineWidth = 3;
    ctx.beginPath(); ctx.moveTo(fBottom.x, fBottom.y); ctx.lineTo(fTop.x, fTop.y);
    ctx.stroke();

    } else if (currentView === 'front') {
        const leftX = toScreen(targetBaseX, safeTargetH, -TGT_W / 2).x;
        const rightX = toScreen(targetBaseX, safeTargetH, TGT_W / 2).x;
        const bottomY = toScreen(targetBaseX, safeTargetH, 0).y;
        const topY = toScreen(targetBaseX, safeTargetH + TGT_PROJ_H, 0).y;
        const w = rightX - leftX;
        const h = bottomY - topY;

        ctx.fillStyle = 'ffffff'; ctx.fillRect(leftX, topY, w, h);
        ctx.strokeStyle = '#1d1d1f'; ctx.lineWidth = 1.5; ctx.strokeRect(leftX, topY, w, h);
        ctx.fillStyle = '#1d1d1f'; ctx.fillRect(leftX + w * 0.1, topY + h * 0.08, w * 0.8, h * 0.15);
        ctx.fillRect(leftX + w * 0.1, topY + h * 0.3, w * 0.8, h * 0.62);
        ctx.fillStyle = '#ff3b30'; ctx.beginPath(); ctx.arc(leftX + w * 0.5, (topY + h * 0.3) + (h *
        0.62) * 0.5, w * 0.23, 0, Math.PI * 2); ctx.fill();

    } else if (currentView === 'top') {
        const projTopX = targetBaseX + TGT_H * Math.sin(TGT_TILT);
        const thickX = TGT_D * Math.cos(TGT_TILT);
        const fLeftBot = toScreen(targetBaseX, safeTargetH, -TGT_W / 2);
        const fRightBot = toScreen(targetBaseX, safeTargetH, TGT_W / 2);
        const bLeftTop = toScreen(projTopX + thickX, safeTargetH, -TGT_W / 2);
        const bRightTop = toScreen(projTopX + thickX, safeTargetH, TGT_W / 2);

        ctx.fillStyle = 'ffffff'; ctx.beginPath(); ctx.moveTo(fLeftBot.x, fLeftBot.y);
        ctx.lineTo(fRightBot.x, fRightBot.y); ctx.lineTo(bRightTop.x, bRightTop.y);
        ctx.lineTo(bLeftTop.x, bLeftTop.y); ctx.closePath(); ctx.fill();
        ctx.strokeStyle = '#1d1d1f'; ctx.lineWidth = 1.5; ctx.stroke();

    } else if (currentView === 'target') {
        const tLeftX = (dprWidth / 2) - (TGT_W / 2 * targetViewScale);
        const tRightX = (dprWidth / 2) + (TGT_W / 2 * targetViewScale);
        const tBottomY = dprHeight * 0.65;
        const tTopY = tBottomY - (TGT_PROJ_H * targetViewScale);
        const w = tRightX - tLeftX;
        const h = tBottomY - tTopY;

```

```

    ctx.fillStyle = '#ffffff'; ctx.fillRect(tLeftX, tTopY, w, h);
    ctx.strokeStyle = '#1d1d1f'; ctx.lineWidth = 2; ctx.strokeRect(tLeftX, tTopY, w, h);
    ctx.fillStyle = '#1d1d1f'; ctx.fillRect(tLeftX + w * 0.1, tTopY + h * 0.08, w * 0.8, h *
0.15);
    ctx.fillRect(tLeftX + w * 0.1, tTopY + h * 0.3, w * 0.8, h * 0.62);
    ctx.fillStyle = '#ff3b30'; ctx.beginPath(); ctx.arc(tLeftX + w * 0.5, (tTopY + h * 0.3) + (h
* 0.62) * 0.5, w * 0.23, 0, Math.PI * 2); ctx.fill();

    if (hasIntersectedTargetPlane) {
        const localYFromBottom = targetHitMetrics.localY + (TGT_PROJ_H / 2);
        const markerX = (dprWidth / 2) + (targetHitMetrics.localZ * targetViewScale);
        const markerY = tBottomY - (localYFromBottom * targetViewScale);

        if (targetHitMetrics.isHit) {
            ctx.fillStyle = '#34c759'; ctx.strokeStyle = 'rgba(52, 199, 89, 0.4)'; ctx.lineWidth =
8;
            ctx.beginPath(); ctx.arc(markerX, markerY, 6, 0, Math.PI * 2); ctx.stroke(); ctx.fill();
            ctx.fillStyle = '#34c759'; ctx.font = 'bold 13px -apple-system'; ctx.textAlign =
'center';
            ctx.fillText("🎯 관중 (HIT!)", dprWidth / 2, tTopY - 14);
        } else {
            ctx.fillStyle = '#ff3b30'; ctx.strokeStyle = 'rgba(255, 59, 48, 0.3)'; ctx.lineWidth = 6;
            ctx.beginPath(); ctx.arc(markerX, markerY, 5, 0, Math.PI * 2); ctx.stroke(); ctx.fill();
            ctx.fillStyle = '#ff3b30'; ctx.font = 'bold 12px -apple-system'; ctx.textAlign =
'center';
            ctx.fillText('❌ 불중 (오차: 좌우 ${targetHitMetrics.localZ.toFixed(2)}m, 바닥높이
${localYFromBottom.toFixed(2)}m)', dprWidth / 2, tTopY - 14);
        }
    }
}

// 지면 지지기둥 기선 드로잉
if (currentView === 'side' || currentView === 'front') {
    const tgtFloor = toScreen(targetBaseX, 0, 0);
    const tgtBasePos = toScreen(targetBaseX, safeTargetH, 0);
    ctx.strokeStyle = '#515154'; ctx.lineWidth = 2; ctx.beginPath();
    ctx.moveTo(tgtBasePos.x, tgtBasePos.y); ctx.lineTo(tgtBasePos.x, tgtFloor.y); ctx.stroke();
}

// [🔗 핵심 버그 패치 완료] 누적 비행 궤적선 변환 공식을 toScreen 시스템으로 강제
단일화 결합
if (currentView !== 'target' && trajectory.length > 1) {

```

```

ctx.strokeStyle = '#0071e3'; ctx.lineWidth = 2.5; ctx.beginPath();
const start = toScreen(trajjectory[0].x, trajectory[0].y, trajectory[0].z);
ctx.moveTo(start.x, start.y);
for (let i = 1; i < trajectory.length; i++) {
  const pt = toScreen(trajjectory[i].x, trajectory[i].y, trajectory[i].z);
  ctx.lineTo(pt.x, pt.y);
}
ctx.stroke();
}

```

// 실시간 화살 오브젝트 렌더링

```

if (currentView !== 'target') {
  const arrowPos = toScreen(arrowState.x, arrowState.y, arrowState.z);
  ctx.save(); ctx.translate(arrowPos.x, arrowPos.y);

```

```

  let angleRad = 0;
  if (currentView === 'side') angleRad = -arrowState.pitch;
  else if (currentView === 'top') angleRad = -arrowState.yaw;
  else if (currentView === 'front') angleRad = Math.atan2(arrowState.vz,
arrowState.vy);

```

```

  ctx.rotate(angleRad);
  ctx.strokeStyle = '#515154'; ctx.lineWidth = 2; ctx.beginPath(); ctx.moveTo(-20, 0);
ctx.lineTo(0, 0); ctx.stroke();
  ctx.fillStyle = '#1d1d1f'; ctx.beginPath(); ctx.moveTo(0, 0); ctx.lineTo(-6, -3);
ctx.lineTo(-6, 3); ctx.closePath(); ctx.fill();
  ctx.fillStyle = '#ff9500'; ctx.beginPath(); ctx.moveTo(-20, 0); ctx.lineTo(-16, -4);
ctx.lineTo(-10, -4); ctx.lineTo(-14, 0); ctx.fill();
  ctx.restore();
}
}

```

// 캔버스 초기 크기 반영 지연 제어

```

setTimeout(() => {
  if (typeof loadSettings === 'function') loadSettings();
  resizeCanvas();

```

```

  const launchH = parseFloat(document.getElementById('launchHeight').value) || 1.5;
// [추가] 초기 Z값 반영
  const launchZ = parseFloat(document.getElementById('launchZ').value) || 0;

```

```

  arrowState.x = 0;
  arrowState.y = launchH;

```

```
arrowState.z = launchZ; // [수정] 기존 0에서 launchZ 변수로 변경!
```

```
    arrowState.pitch = (parseFloat(document.getElementById('angle').value) || 30) *  
Math.PI / 180;
```

```
    arrowState.yaw = (parseFloat(document.getElementById('yawAngle').value) || 0) *  
Math.P
```

```
    drawScene();  
}, 250);
```